Noise Reduction – Generation 2

Software Users Guide

Adaptive Digital Technologies, Inc.

August 12, 2012
Issue 4

Copyright © 2008 – 2019 Adaptive Digital Technologies Inc.
Proprietary

Revision History

Issue	Revision History	Date	Initials
1	V2_00	05/16/2008	HFZ
2	V2.01 – Added ability to control noise reduction aggressiveness	08/19/2011	SDK
3	V2_02 – a bug in the noise reduction aggressiveness calculation was fixed	09/02/2011	SDK
4	V3_01, C55X optimization	08/12/2012	HFZ

Table of Contents

1. Introduction 1
2. Noise Reduction 1
2.1 Instantiation 1
2.2 Initialization 1
2.2.1 Initialization API 1
2.2.1.1 Default Initialization 1
2.2.1.2 Parametric Initialization 2
2.2.2 Get Library Version 2
2.2.3 Reduce Function 2
2.3 Sample Host Code 3

1. Introduction

This document describes the Adaptive Digital second-generation noise reduction software.

The Adaptive Digital noise reduction software reduces background noise in speech signals. Applications include traditional, mobile, and hands-free telephone systems, conferencing, speech recognition, and alarm systems.

2. Noise Reduction

ADT's Noise Reduction software is designed using object-oriented methodology in order to achieve a multiple channel architecture.

2.1 Instantiation

The noise reduction algorithm requires memory to be allocated for each channel. The following example shows how to instantiate a data for a single channel.

```
#include "nradt_user.h"

NRADT_Scratch_t Scratch;                  //declare scratch space
NRADT_Channel_t NR_Channel;               //declare a single instance
```

2.2 Initialization

Each time a new circuit is established, the noise reduction associated with that circuit must be initialized.

2.2.1 Initialization API

The user may choose between two different APIs for initialization. The first one (NR_ADT_init) has no configurability, but is available for backward compatibility. The second one (NR_ADT_initWithParam) allows the user go configure the noise reduction algorithms aggressiveness.

2.2.1.1 Default Initialization

The following is an example of a parameters structure that is used to initialize the noise reduction.

Prototype

```
void NR_ADT_init (NRADT_Channel_t *Channel,
                  NRADT_Scratch_t *pScratch)
```

Inputs:

Channel – pointer to the per-channel persistent instance data
pScratch – pointer to scratch data

Outputs:

None

2.2.1.2 Parametric Initialization

The following is an example of a parameters structure that is used to initialize the noise reduction.

Prototype

```
void NR_ADT_initWithParam (NRADT_Channel_t *Channel,
                           NRADT_Scratch_t *pScratch,
                           ADT_Int16 ReductionAmount)
```

Inputs:

Channel – pointer to the per-channel persistent instance data
pScratch – pointer to scratch data
ReductionAmount – A measure of how aggressively the algorithm should reduce noise. The range is from 0 (no noise reduction) to 32 (maximum noise reduction)

Outputs:

None

2.2.2 Get Library Version

Prototype:

char *NR_ADT_getLibraryVersion(void)

Inputs: None

Outputs: None

Returns: pointer to string that contains the library build version

2.2.3 Reduce Function

The reduce function does all the work of the noise reduction. This function is called with a frame's worth of data where the frame size is specified by the calling function.

Prototype:

```
void NR_ADT_reduce(NRADT_Channel_t *Channel,
                   ADT_Int16 Input[],
                   ADT_Int16 Output[],
                   ADT_Int16 Length)
```

Inputs:

Channel – pointer to noise reduction per-channel data
Input [ ] – array of Length input samples, 16 bit PCM
Output [ ] – array of Length output samples, 16 bit PCM
Length – length of input and output arrays, maximum frame size is 192 samples

2.3 Sample Host Code

```
#include "adt_typedef_user.h"
#include "nradt_user.h"
NRADT_Scratch_t Scratch;
NRADT_Channel_t NR_Channel;

Void init( )
{
      NR_ADT_init (NR_Channel, Scratch);
}


Void ProcessData(short int Input[ ], short int Output[ ], short
                  int Length)
{
…
      NR_ADT_reduce(NR_Channel, Input, Output, Length);
…
}
```